Introduction

Our names are Isaiah and Joshua. For our design project in EE230 we chose to do the Signal Generator. The following specs and conditions describe what needed to be done for our project. We relied significantly on LTSpice to create and test our circuits virtually first and allowed us to complete this project.

- Circuit must have no inputs (excluding op amp rails)
- Circuit must have a potentiometer to tune the frequency in a range of 500Hz - 5kHz
- Can use a max of two DC power supplies

- Square Wave: Magnitude peaks at ± 5V
- Triangle Wave: Magnitude peaks at ± 2V
- Sin Wave: Magnitude of **at least** 1V **at any frequency**

Description of Design and Design process

Design / Design processes (including equations used)

Brainstorming and finding circuits in the textbook and online to replicate in LTSpice and on our breadboards would be easy, but using only 1 potentiometer to control all 3 waveforms was not. This made the design project significantly harder. We blindly experimented in LTSpice with circuits we found online and in the text book with different combinations of values to see if we could get close with the individual circuits themselves. Our idea was to try to implement each circuit individually, and then try to link them together with the resistor potentiometer.

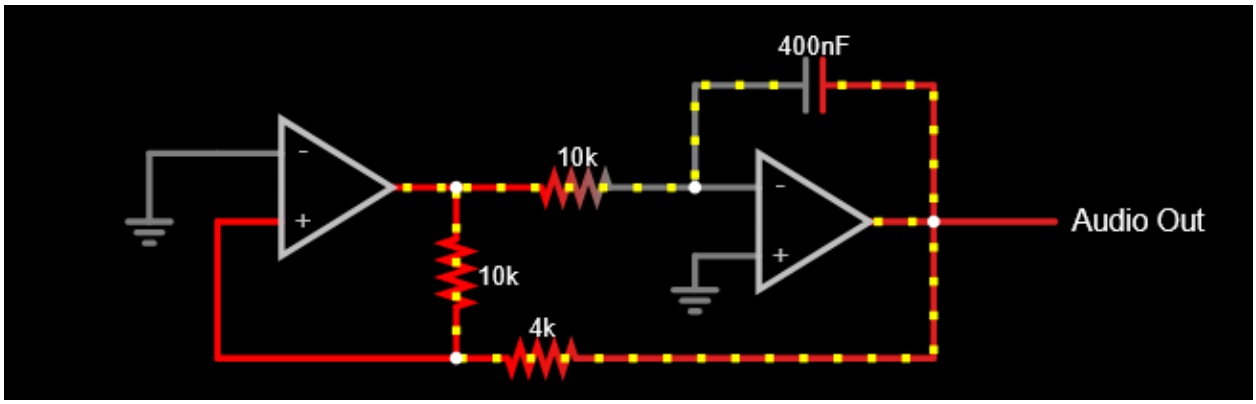
But we quickly found out this idea was severely flawed as that meant the signals in each circuit would be connected and interfere with each other as they went to the potentiometer.

This led us to build, edit, and test what would be our final circuit for the signal generator design project.

Our final circuit started with a triangle wave generator with the potentiometer resistor embedded into it. This latter aspect allowed the frequency to change for the triangle waveform as well as everything downstream (aka the sine wave and square wave generators). We used the triangle wave generator and converted the signal to a square wave, and the square wave to get our sine wave. This is how we got our 3 wave forms. How we implemented them will be discussed below.

Triangle generator:

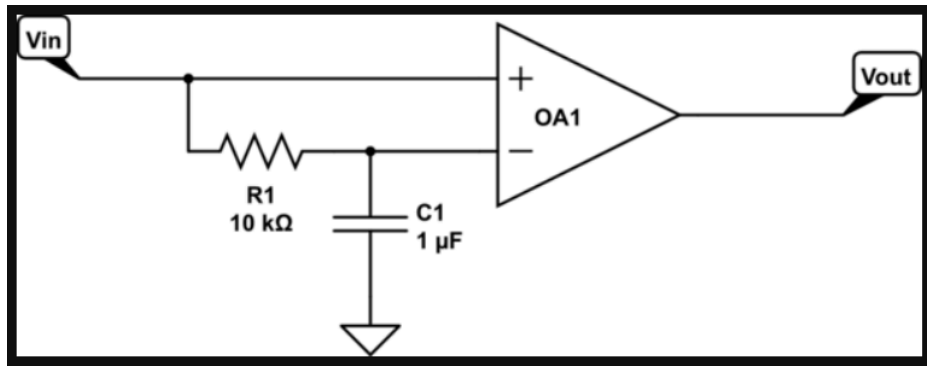
We searched the internet for a triangle generator circuit, eventually finding a website by Paul Falstad showcasing the following circuit.



The triangle wave generator uses two op amps to act as a comparator and an integrator circuit. The comparator acts as a Schmitt trigger, switching its output between high and low states when the input crosses certain thresholds. Its output drives the integrator, which produces a steadily rising and falling voltage, forming a triangle wave. The triangle wave is then fed back into the comparator's input, causing the circuit to continuously oscillate. This causes a stable triangle waveform to occur. When implementing a potentiometer in the location of the bridging resistor, it allows for manipulation of the frequency of the triangular waveform. Although the screenshot shows resistor and capacitor values, the bridging resistor became a 100k potentiometer and the capacitor became 10nF.

Square generator:

We searched the web for circuits that could convert triangle waveforms into a square waveform. We stumbled upon [this site]() that showed how a comparator could be used to convert a triangle wave into a square wave. The circuit shown already had values for its capacitor and resistor, so we immediately implemented it as is and it worked perfectly; the amplitude was ±5V and the frequency the same as the triangle wave input. After tracing through it and reasoning, it does make sense intuitively.



Node Voltage at V_n:

$$\frac{V_n - V_i}{10k\Omega} + \frac{V_n - 0}{1/sC} = 0 \rightarrow \left[V_n\left(\frac{1}{10k\Omega} + \frac{1}{(1/sC)}\right) = \frac{V_i}{10k\Omega}\right] \cdot 10k\Omega$$

$$= V_n(1 + 10ksC) = V_i \rightarrow \frac{V_n}{V_i} = \boxed{\frac{1}{1 + 10ksRC} = H_{RC}(s)}$$

Comparator: non_inv. − inv.

$$V_{out} = \begin{cases} V_{High}, & V_i(t) - V_n(t) > 0 \\ V_{Low}, & V_i(t) - V_n(t) < 0 \end{cases}$$

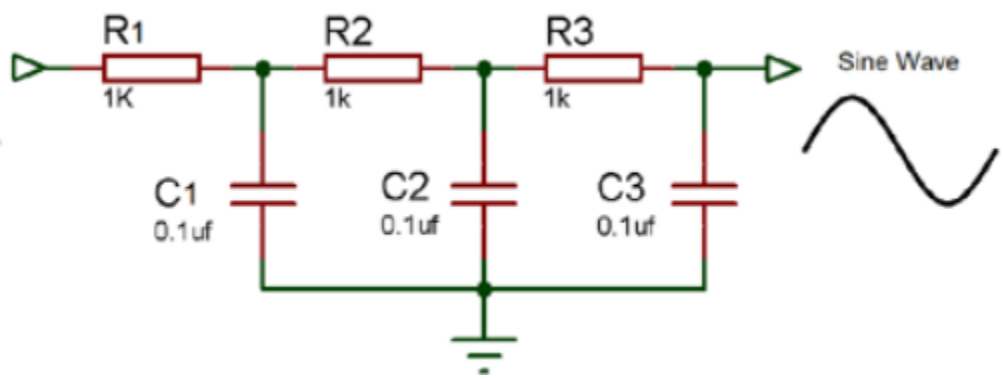
There is no feedback loop, so whatever difference it sees between the two inputs, it saturates the output to the positive or negative rail. One input (+) gets the actual triangle wave. The other input goes through an RC circuit. This shifts the phase, shifting the triangle wave over.

The capacitor cannot keep up with the triangle wave changing direction. When the triangle is going up, the filtered version is always a little behind it. When the triangle is going down, the filtered version is a little ahead. The comparator is looking at whether the triangle is rising or falling. If the triangle is rising, the main input is higher than the lagged version, so the comparator output goes high. This is the opposite of when the output goes low.

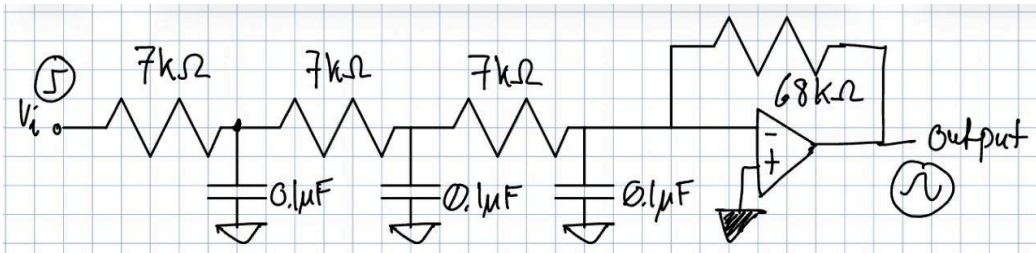
We also tried to do calculations, but weren't sure how to check using our calculations and a comparator.

Sine generator:

We also searched the web for circuits that could convert a triangle waveform into a sine waveform. We tried to implement a few different ones but they didn't work. We attempted just a resistor and amplifying it but that didn't work. We tried attaching a separate oscillating sine generator and that didn't work. We tried a resistor followed by two opposing diodes on separate lines and ground on either side and that didn't work either. So we turned our focus from converting a triangle wave to a sine wave to converting a square wave to a sine wave instead.



We eventually found the diagram shown on the left of a 3-pole low-pass filter. We implemented it, experimented with different values, and checked the waveform when the pot was at 10kΩ and 100kΩ. We found that decreasing the values of R1, R2, and R3 yielded a better sine wave at high frequencies, but resulted in the sine wave turning back into a more pronounced square wave. We also found that turning the resistor values up higher yielded a much better sine wave at lower frequencies, but a sine wave with an extremely small magnitude at higher frequencies, even after being amplified. We tried to find a middle ground for the resistor values using LTSpice. The simulation showed a not too great sine waveform at ~100kΩ, but during the demo it turned out much better than expected. The results we got from the demo looked very promising and satisfied us despite LTSpice showing us that it wouldn't work for the range we were hoping for. We worked through the calculations as well:



$$H_1(s) = \frac{1}{1 + sRC}, \quad f_c = \frac{1}{2\pi RC}, \quad R = \frac{1}{2\pi f_c C}$$

$$|H_1(s)| = \frac{1}{\sqrt{1 + (2\pi f_0 C (R_{source} + R))^2}}$$

- R_{source}: Resistance between Square Wave output node & Sine Wave input node

Cascading:

$$G = (|H_1|)^3 = \left[\frac{1}{\sqrt{1 + (2\pi f_0 C (R_{source} + R))^2}}\right]^3$$

$$R = \frac{1}{2\pi f_0 C} \cdot \sqrt{G^{-2/3} - 1} - R_{source}, \quad R_{source} = 0$$

for 500 Hz: let $G = 0.0709 \rightarrow R \approx 7{,}001\,\Omega$

for 5,000 Hz: let $G = 93.74 \times 10^{-6} \rightarrow R \approx 6{,}999\,\Omega$

let $R = 7{,}000\,\Omega$ given different values of gain b/c freq. ranges.

also given $C = 0.1\mu F$

where do G values come from?:

$$\tau = RC = 7k \times 0.1\mu F = 7 \times 10^{-4}\,s$$

$$f_c = \frac{1}{2\pi\tau} \approx \frac{1}{2\pi \cdot 7 \times 10^{-4}} \approx 227\,Hz$$

Pole for 1 RC

$$|H_1| = \frac{1}{\sqrt{1 + (f/f_c)^2}}, \quad G = |H_1|^3$$

at 500 Hz: $\alpha = f/f_c = \frac{500}{227} \approx 2.20$

$$|H_1| = \frac{1}{\sqrt{1+\alpha^2}} = \frac{1}{\sqrt{1+2.20^2}} \approx \frac{1}{\sqrt{5.84}} \approx 0.414$$

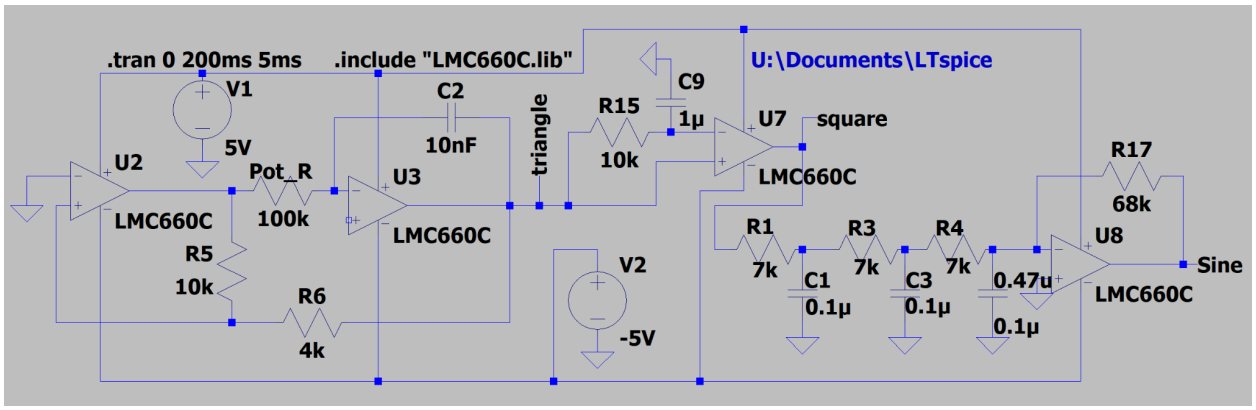
$$G = 0.414^3 = 0.0709.$$

at 5kHz: $\alpha = \frac{f}{f_c} = \frac{5000}{227} \approx 22.0$

$$|H_1| \approx \frac{1}{\sqrt{1+22^2}} = \frac{1}{\sqrt{485}} \approx 0.0455$$

$$G = 0.0455^3 \approx 9.374 \times 10^{-5} = 93.74\mu$$

Complete Schematic of Final Circuit (with component values):



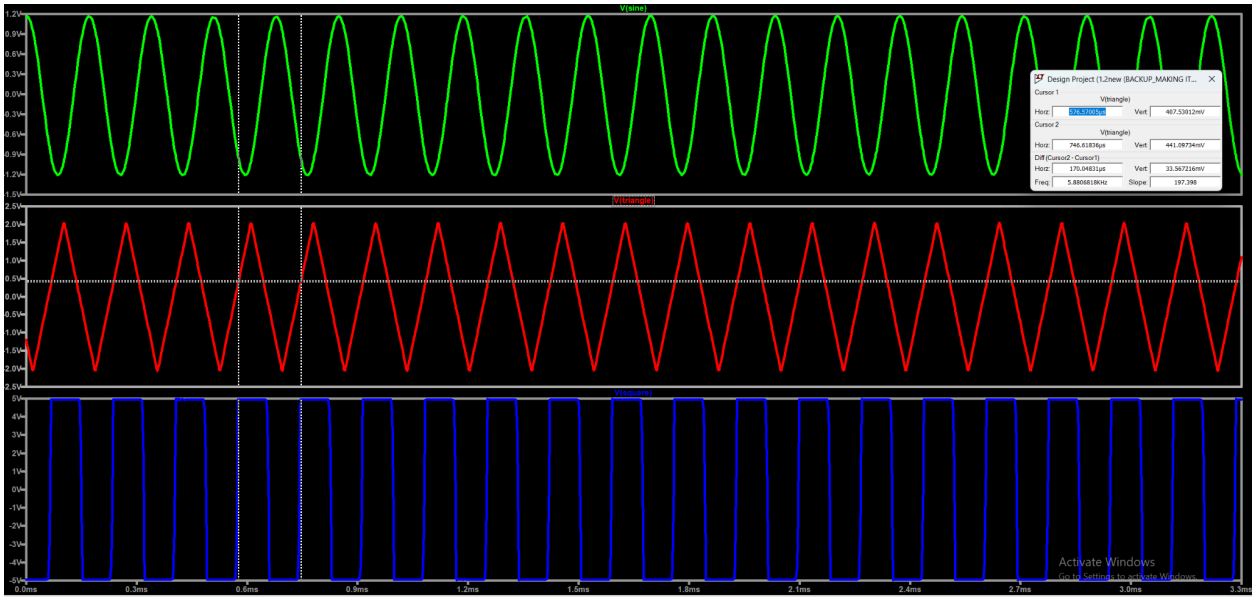
Description of simulations performed (LTSpice waveforms):

The simulations we ran were by adjusting the Pot_R resistor between two values; ~10kΩ and ~100kΩ. We got the wave forms on either end and screenshotted them along with the windows giving the frequencies.

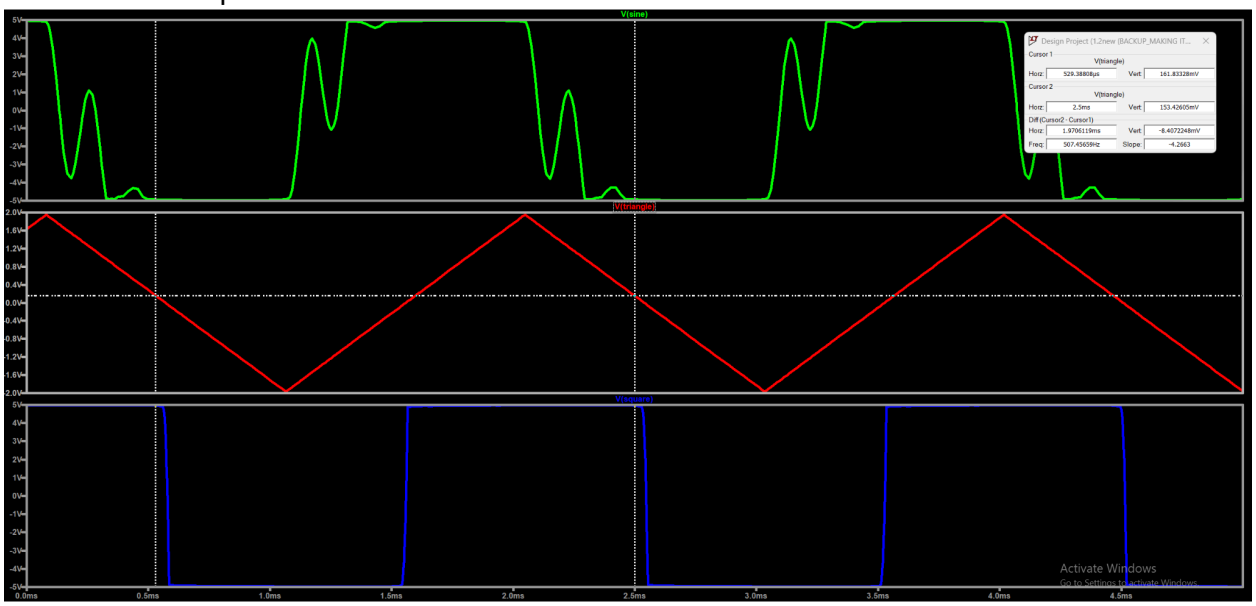
The waveforms generated for the ~10kΩ looked great and met the required specs. The ~100kΩ range was close to meeting the required specs, if not arguably there. Everything looks good except for the sine wave. We explained above that adjusting the resistor values helped us achieve a good looking sine wave, but not for the entire range of ~10kΩ to ~100kΩ. It is also weird because the actual waveform generated with our real circuit looked much better than on LTSpice. We are not sure why it is, or if more non-ideal parts of the elements in the circuit actually helped us instead of hurting us. We are not sure. Although there are some other inconsistencies with LTSpice as well; in the lab our potentiometer was able to get the circuit to much higher and lower frequencies before becoming unstable as compared to LTSpice. LTSpice's waveforms become much more unstable much faster at smaller ranges of values.

Simulation Results

SINE WAVE SQUARE WAVE TRIANGLE WAVE

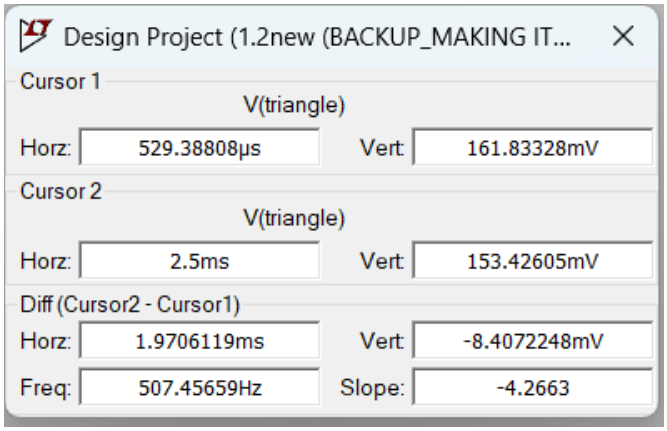


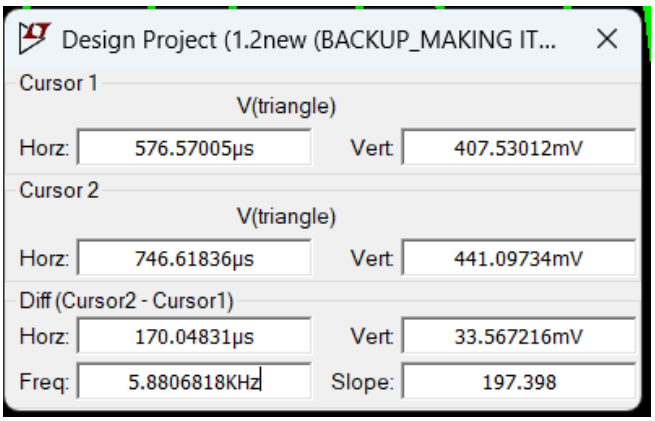
This is when the potentiometer is at ~10kΩ



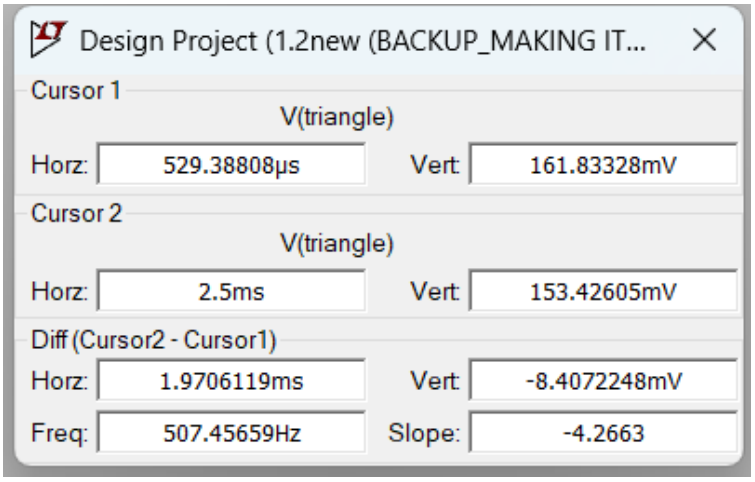
This is when the potentiometer is at ~100kΩ

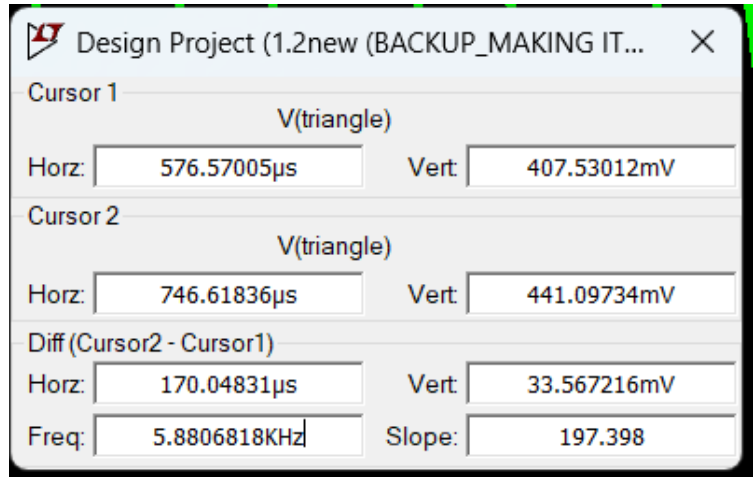
Left is ~100kΩ Right is ~10kΩ





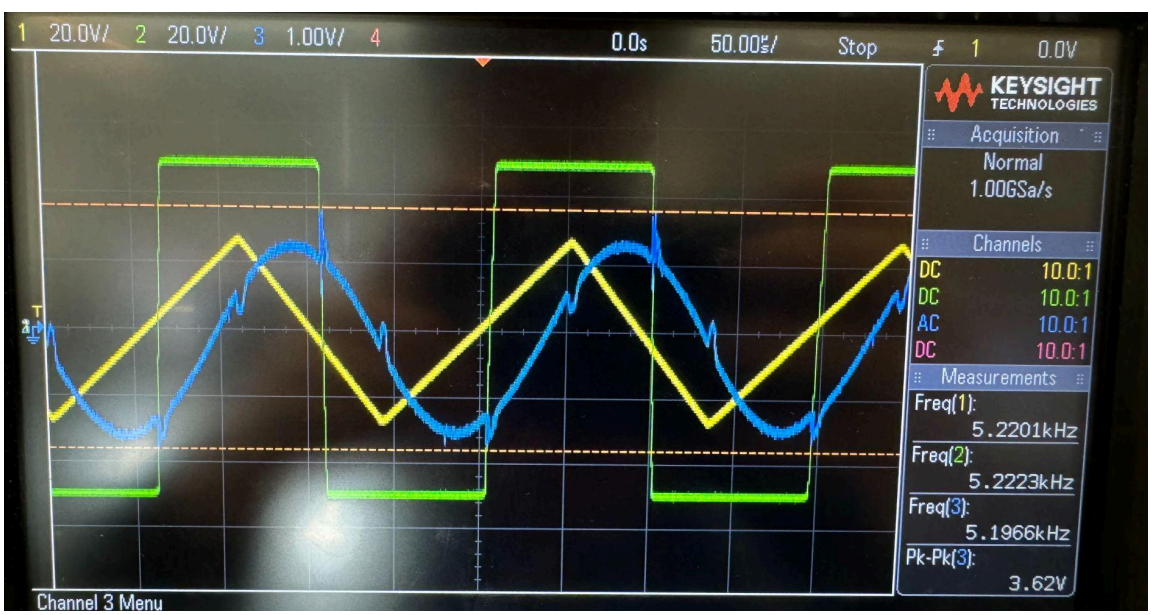
Required Measurements from project description



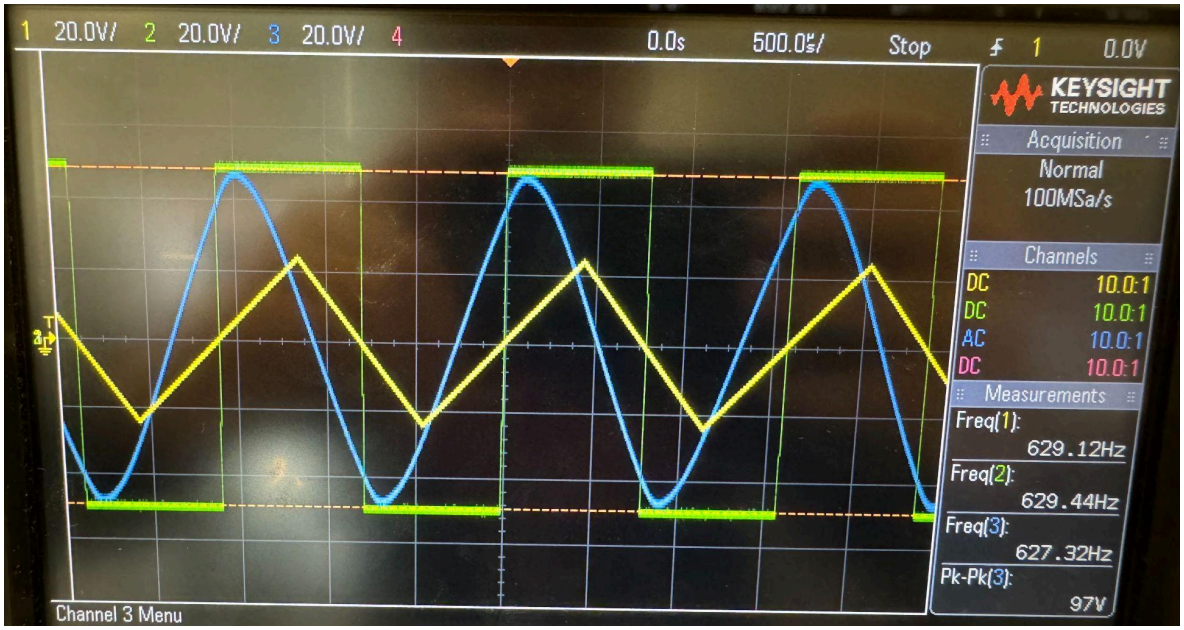


Left measurement shows frequency when potentiometer is at ~100kΩ

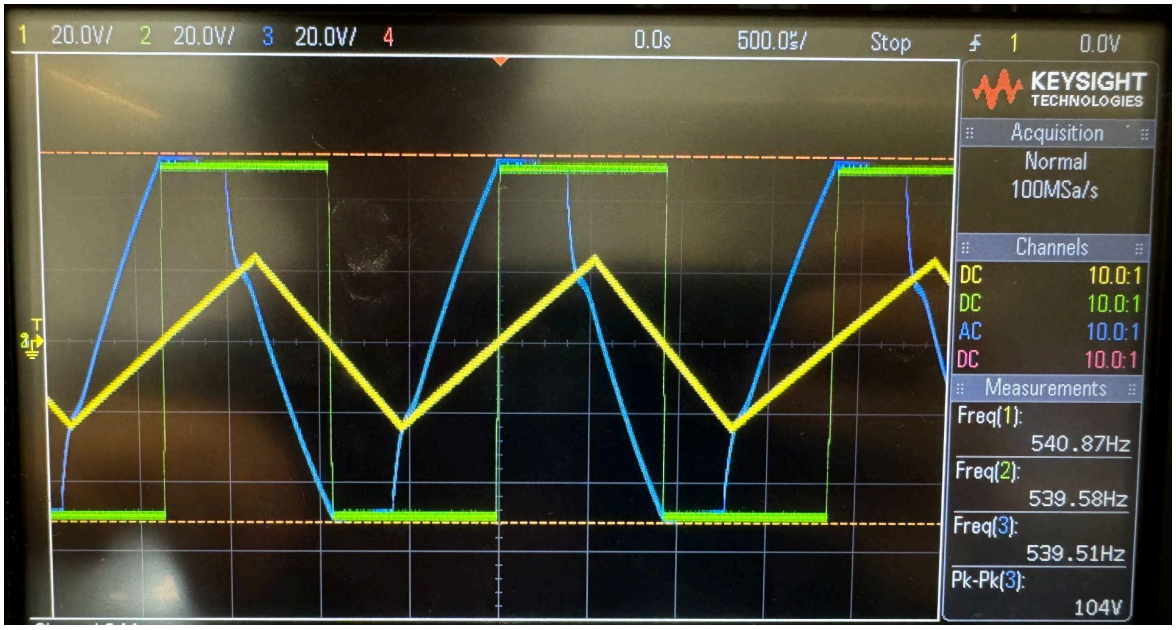
Right measurement shows frequency when potentiometer is at ~10kΩ



This is our waveform at the 5kHz range

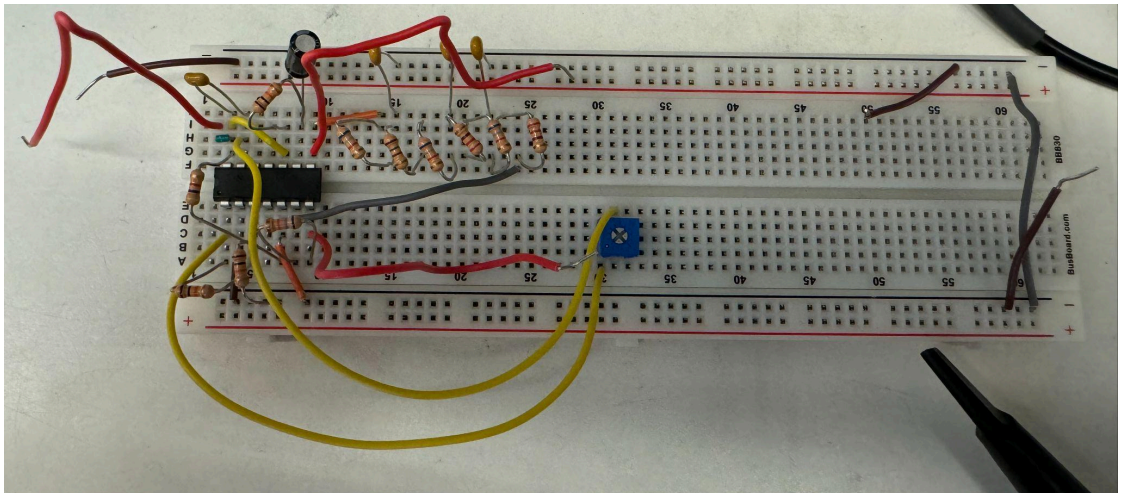


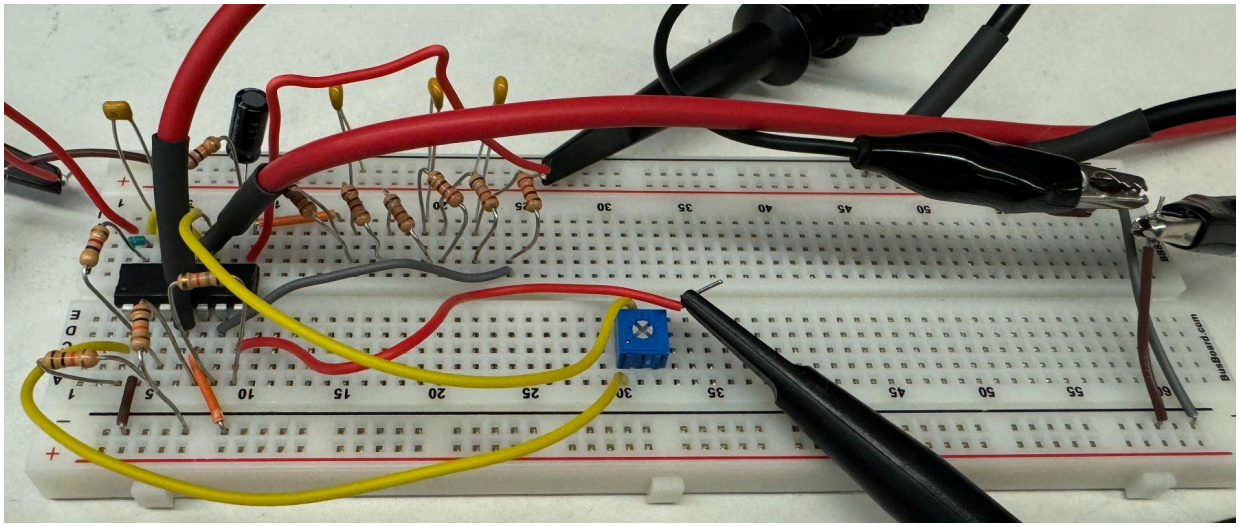
This is our waveform at the lowest frequency it can go without the sine wave clipping

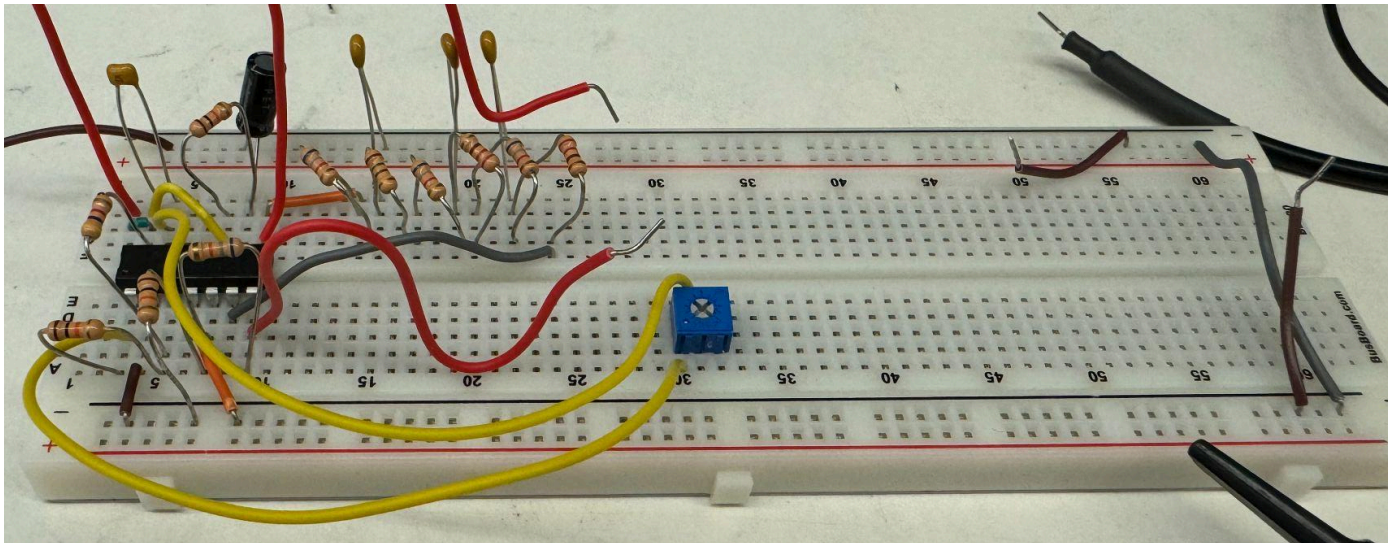


This is our waveform at the lowest frequency it can go

Photograph of Circuit







Conclusion

In this design project we successfully created a fully functional signal generator capable of producing a square wave, triangle wave, and sine wave simultaneously with a tunable frequency range using a single potentiometer. The final circuit met all design goals:

The triangle wave oscillated symmetrically at ± 2 V, the square wave reached peaks of ± 5 Vm the sine wave maintained a magnitude of at least 1 V, all throughout the usable and required frequency range of 500Hz to 5kHz.

Creating our final circuit and hitting the criteria listed required lots of time testing and iterating different parts of the circuit in LTSpice and the refinement of the individual circuits of the larger total waveform generator.

One of the biggest challenges we faced was designing the sine wave generator, followed closely by using only 1 potentiometer resistor to control the frequency of all 3 waveforms. Earlier implementations wouldn't work how we wanted them too. 1 circuit worked on its own but we didn't know how to incorporate it or the variable resistor. Another one just didn't do a good job converting the signal. Another implementation made the input signal far too weak and got reverted back to the input waveform after amplifying it. What we finally got working was a cascading RC filter that mostly works; it works better at some frequencies than others. But for the vast majority of the range [5kHz, 500Hz] it works fantastic.

The next biggest challenge as mentioned above was designing a system where all three waveforms were linked and controlled by the single potentiometer without having the circuits interfere with each other. Early attempts involved building each waveform circuit separately and we had planned to link them together in parallel at the end. But this would mean the circuits had to connect to each other which would interfere greatly with the signals, and it did. We ultimately settled on a design where we would make an oscillator with an output that we needed, and use

that output to push our other two wave forms. Implementing the circuit this way allowed us to put our potentiometer earlier in the circuit and affect that waveform in addition to the other two waveforms downstream. It is also useful for debugging because you can see which waveform(s) are wrong or messed up and deduce around where in the circuit the errors are being caused.

Simulating iterations of the final circuit allowed us to hone in on our now final values for different elements of the circuit. Although the real hardware often behaved more favorably than the simulations on LTSpice had predicted. This is especially true for the sine-wave filter. LTSpice will sometimes lower tolerances to continue computing the expected output. Sometimes it just straight up quits. But the real hardware might continue to output the waveform just with some minor clipping (see sine waveforms in first and last trace images).

Overall, this project strengthened our understanding of op-amp behaviour, waveform shaping, filtering, and a large difference between idealized simulation models and real components, real error, real inconsistencies, etc. We also gained experience with debugging non-ideal circuits, tuning component ranges, and ensuring stability of the waveforms across a large range of frequencies. Despite the complexity of converting one waveform into a different one, and using only one potentiometer to alter the frequency of every waveform, our final design performed reasonably reliably in both simulation and hardware demonstration. It satisfied all the requirements outlined in the assignment specs. This project allowed us to apply concepts we have learned throughout EE230 to a real, functional circuit that designed, tested, built, and tested again, all from the ground up.

Sources (bibliography)

Square Wave:

Triangular waveform to square waveform circuit - Electrical Engineering Stack Exchange

Sine Wave:

Square Wave To Sine Wave Converter Circuit Diagram

Triangle Wave:

https://www.falstad.com/circuit/e-triangle.html